SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UET106 | Name: Nihal Ambi | Division: B | Date: 12 October 2026

PART A: Architecture Decision Record (ADR)

Title: Use Breadth-First Search (BFS) as the Main Solver for Maze Path-Finding
Status: Accepted | Date: 12 October 2026 | PRN & Name: 25UET106 – Nihal Ambi

1. Context

1. The project compares BFS and Depth-Limited DFS for maze path-finding.
2. The maze is a solvable grid with the start at the top-left and the goal at the bottom-right.
3. The search uses BFS and depth-limited DFS with a limit of 1000.
4. The SLE-2 repository records tests on 20×20, 40×40 and 70×70 mazes. It measures execution time and expanded nodes.
5. The SLE-3 repository explains the system architecture using the C4 model.

2. Decision

I chose BFS as the main solver when the shortest path is required. DFS remains in the project for comparison. The SLE-2 README states that BFS found the true shortest path in the tested mazes, while DFS did not.

3. Alternatives Considered

Depth-Limited DFS (limit 1000): used as the comparison algorithm. In the reported tests, it was faster and expanded fewer nodes, but it did not find the shortest path.

BFS: selected as the main solver because it returned the shortest path in the reported test cases.

4. Rationale / Justification

The values below are taken from the SLE-2 repository README. The README notes that exact figures can vary between runs because mazes are randomly generated.

Maze size	BFS steps	DFS steps	BFS nodes	DFS nodes
20×20	38	72	295	155
40×40	78	166	1,212	674
70×70	138	390	3,739	2,421

BFS returned shorter paths in all three reported cases.

DFS expanded fewer nodes in the recorded tests. This depends on the maze wall layout and the fixed neighbour order (up, down, left, right); it does not mean DFS is always better.

The SLE-3 architecture lists Maze Generator, Search Engine, Path Reconstructor, Output & Metrics, and Experiment & Profiling. The Search Engine includes Frontier, Visited Set, Goal Test, Parent Map, BFS and DFS.

5. Consequences

Positive:

BFS is complete and optimal on an unweighted maze, as stated in the SLE-2 README.

The project can compare path length, expanded nodes and execution time for BFS and DFS.

Negative / trade-off:

BFS expanded more nodes than DFS in the reported test cases.

The SLE-2 README gives BFS time and space complexity as O(b^d).

6. References

SLE-1/ IAI: https://github.com/nihalambi32/IAI
SLE-2: https://github.com/nihalambi32/SLE-2
SLE-3: https://github.com/nihalambi32/SLE-3

PART B: Stepwise Summary of SLE-1, SLE-2 and SLE-3

SLE-1: Student AI Assistant (IAI repository)

Concept: A simple rule-based conversational program written in Python. It responds to specific student-related keywords with prepared answers. It is not described in the README as a trained machine-learning model.

1. Open the program: the main function is student_agent() in ai_agent.py.
2. The assistant displays its title and tells the user to type “bye” to exit.
3. The user types a message. The program converts the message to lowercase.
4. The program checks the message for known words or phrases, such as hello/hi, python, DSA/data structure, exam, college or study.
5. When a known keyword is found, it prints the matching prepared response.
6. When the user types “bye”, the program prints a goodbye message and stops.
7. If no known keyword matches, the program prints that it does not have an answer.
8. GitHub Copilot was used to generate or modify selected code sections. The README says suggestions were reviewed, tested and modified where needed; the contribution log records the selected contributions.

Technologies listed in the README: Python 3.12+, Visual Studio Code, Git, GitHub and GitHub Copilot. The repository contains ai_agent.py, README (1).md and Contribution_Log(2).md.

SLE-2: BFS vs DFS Maze Profiling

9. Generate a solvable grid maze with a start and goal.
10. Run BFS and Depth-Limited DFS to search for a path.
11. Repeat the experiments on 20×20, 40×40 and 70×70 mazes.
12. Compare the path length, expanded nodes and execution time. The repository mentions time.perf_counter and py-spy for profiling.
13. Reported result: BFS found shorter paths; DFS expanded fewer nodes and was faster in the recorded tests.

SLE-3: C4 Architecture

14. Use the C4 model to describe the structure of the BFS/DFS maze project.
15. The four C4 levels are Context, Container, Component and Code.
16. The documented components are Maze Generator, Search Engine, Path Reconstructor, Output & Metrics, and Experiment & Profiling.
17. The Search Engine includes Frontier, Visited Set, Goal Test, Parent Map, BFS and DFS.

PART C: AI Contribution Note

18. The AI coding tool used in SLE-1was GitHub Copilot.
19. The IAI README says Copilot helped generate or modify selected parts of the Python program.
20. The contribution log records suggestions for the student_agent() structure, the while loop, bye exit, greeting, Python, DSA, study advice and default response.
21. The student reviewed, tested or modified the suggestions rather than accepting the whole program without understanding it.